Enhanced LED Control

Youso Lee

Submitted
April 21, 2025

Introduction

This document summarizes our Enhanced LED Control peripheral project for the DE10 board, highlighting the development process, functionality, and outcomes of our efforts. Initially, the DE10 board offered limited LED control, primarily restricted to basic on-and-off states. Our peripheral extends these capabilities by providing customizable brightness levels, dynamic lighting patterns, and precise control of individual LEDs through an intuitive peripheral API. Our team prioritized simplicity, ease-of-use, and clear demonstration of functionality. Although we considered implementing interactive demonstrations like games, we ultimately selected a straightforward "Christmas light" example to clearly showcase the peripheral's features. We successfully met all outlined requirements and project goals, resulting in a robust and user-friendly device.

Device Functionality

State Functionality and Register Input

Our peripheral operates using a 16-bit register through direct user input. Bits 15 and 14 determine the desired state. According to our VHDL definitions, "00" indicates off, "10" means the LED is on with brightness control enabled, "11" signals a default flashing behavior with adjustable brightness, and "01" represents an unused but clearly defined state reserved for potential future use. Bits 13 through 10 determine the brightness level when activated, ranging from 0 (off) to 15 (maximum brightness), providing 16 discrete brightness levels per LED. Bits 9 through 0 specify which LED is affected by the user's input, enabling precise individual LED control via assembly code.

```
ENTITY LEDController IS
PORT(
    CS          : IN  STD_LOGIC;
    WRITE_EN    : IN  STD_LOGIC;
    RESETN      : IN  STD_LOGIC;
    LEDs        : OUT STD_LOGIC_VECTOR(9 DOWNTO 0);

    IO_DATA     : IN  STD_LOGIC_VECTOR(15 DOWNTO 0);
    CLOCK       : IN  STD_LOGIC
    );
END LEDController;

ARCHITECTURE a OF LEDController IS
    -- Using state bits directly instead of enumerated type
    -- state_bits(1 downto 0):
    -- "00" -> OFF
    -- "10" -> ON (with PWM brightness)
    -- "11" -> ON (with flashing and PWM brightness)
    -- "01" -> Undefined (treated as OFF)

    -- Arrays to track state and brightness of each LED
    TYPE led_state_array IS ARRAY (0 TO 9) OF STD_LOGIC_VECTOR(1 DOWNTO 0);
    TYPE led_brightness_array IS ARRAY (0 TO 9) OF INTEGER RANGE 0 TO 255;
```

Figure 1. Peripheral VHDL Entity List and Architecture

Pulse Width Modulation (PWM) Brightness Control

To simulate decreased light intensity, we employed Pulse Width Modulation (PWM). PWM uses a constant counter, allowing the LED to remain on only when the counter value is less than the inputted brightness level. This rapid on-and-off cycling effectively creates varying brightness levels imperceptible to the human eye. Resetting the counter is crucial for maintaining continuous PWM functionality, enabling direct, logical LED brightness control.

```
        IF RISING_EDGE(CLOCK) THEN
            IF pwm_counter = 255 THEN
                pwm_counter <= 0;
            ELSE
                pwm_counter <= pwm_counter + 1;
            END IF;
        END IF;
    END PROCESS;

    -- Process to generate LED outputs based on states and brightness
    LED_OUTPUT_PROCESS: PROCESS (CLOCK)
    BEGIN
        IF RISING_EDGE(CLOCK) THEN
            FOR i IN 0 TO 9 LOOP
CASE led_states(i) IS
    WHEN "00" => -- OFF
        led_outputs(i) <= '0';

    WHEN "10" => -- ON with PWM brightness
        IF pwm_counter < led_brightness(i) THEN
            led_outputs(i) <= '1';
        ELSE
            led_outputs(i) <= '0';
        END IF;

    WHEN "11" => -- ON with flashing and PWM brightness
        IF flash_state = '1' THEN
            IF pwm_counter < led_brightness(i) THEN
                led_outputs(i) <= '1';
            ELSE
                led_outputs(i) <= '0';
            END IF;
        ELSE
            led_outputs(i) <= '0';
        END IF;

    WHEN "01" => -- Undefined state (treated as OFF)
        led_outputs(i) <= '0';

    WHEN OTHERS => -- Shouldn't happen, but safety first
        led_outputs(i) <= '0';

END CASE;
```

Figure 2. VHDL Code for Pulse Width Modulation

Gamma Correction

Gamma correction enables distinct and noticeable brightness levels by squaring the input brightness level. This mathematical approach creates a nonlinear brightness curve, providing a significant and easily discernible difference between brightness levels. Squaring the brightness input is both straightforward and accurate.

```
IF IO_DATA(i) = '1' THEN
    -- Update state based on bits [15:14]
    led_states(i) <= IO_DATA(15 DOWNTO 14);

    -- Update brightness based on bits [13:10]
    led_brightness(i) <= TO_INTEGER(UNSIGNED(IO_DATA(13 DOWNTO 10))*UNSIGNED(IO_DATA(13 DOWNTO 10)));
END IF;
```

Figure 3. VHDL Code For Gamma Correction

Design Decisions and Implementation

Our primary goal was simplicity and creating an intuitive, user-friendly API. Initially, we explored creative demonstration methods, such as interactive games like Rock, Paper, Scissors or Simon Flash. However, recognizing the complexity and potential confusion these might introduce, we ultimately chose to explicitly demonstrate each feature. This decision emphasized the peripheral's flexibility and ease of control without sacrificing clarity.

Conclusions

Our project successfully achieved its primary objective of enhancing LED control on the DE10 board, particularly through features like PWM and gamma correction. Despite encountering challenges during the development process, effective teamwork allowed us to overcome obstacles and complete the project as intended. For future development, we recommend further refinement of the API to facilitate integration into more complex applications. Additionally, an earlier start and iterative testing would've optimized the design process. Reflecting on our experience, we learned the importance of balancing creative functionality with user clarity and simplicity, an approach we believe will greatly benefit future engineering efforts.